

Computer Programming

Final Project

Airline Booking System

(Cloud2GO)

Group Leader

John Enrick Yumul

Members

Jonhval Patrick Collado

Rouel James Caldoza

Mark Christian Valenzuela

Section 1: Introduction

1.1 Project Overview & Target SDG

CLOUD2GO Airline Booking System is a console application for managing airline reservations created using C++ programming language. It offers several functions to help users book flights and monitor their booking process by such functions as view flights, reserve seats, manage passenger data, cancel booking, record transactions, and monitor booking history. Concepts used in this program include functions, classes, vectors, files, and data validation that help organize a user-friendly reservation system.

The purpose of creating this application is offering an opportunity to manage airline reservations more effectively and accurately compared to manual booking transactions by improving the efficiency of managing airline records using computer programming concepts. The project relates to Sustainable Development Goal (SDG) 9 – industry, innovation, and infrastructure since it contributes to innovation in the field of transport services through the use of technology and software.

1.2 Problem Statement

Booking flights manually is an often tedious, unorganized task that may lead to human mistakes, such as wrongly entering passenger details, double-booking seats, and misplacing records for passengers. Organizing the transactions of passengers manually would make it difficult to manage booking history and updating information about passengers effectively.

Section 2: Requirements Analysis

2.1 Functional Requirements

FR1: The system must display all available flights with corresponding destination and ticket price.

- Seen in: showFlights()

FR2: The system must allow users to search for flights based on their budget.

- Seen in: searchByBudget()

FR3: The system must allow passengers to book flights and reserve seats.

- Seen in: processBooking()

FR4: The system must store booking and transaction records.

- Seen in: history vector, transactions vector, saveData()

FR5: The system must display transaction history and booking receipts.

- Seen in: viewHistory()

FR6: The system must allow administrators to cancel bookings.

- Seen in: cancelBooking()

FR7: The system must allow passengers to update their personal information.

- Seen in: updatePassengerInfo()

FR8: The system must allow passengers to change reserved seats.

- Seen in: updatePassengerSeat()

FR9: The system must save and load flight data using an external database file.

- Seen in: loadData(), saveData()

FR10: The system must allow users to exit the application safely.

- Seen in: main()

2.2 Non-Functional Requirements

NFR1: Input Validation

- The system should prevent invalid inputs by validating integer and decimal values.

- Seen in: getValidatedInt(), getValidatedDouble()

NFR2: Reliability

- The system should correctly save and retrieve booking information from the database file.

- Seen in: loadData(), saveData()

NFR3: Usability

- The system should provide a simple console-based menu for easy navigation.
- Seen in: main() menu interface

NFR4: Readability

- The source code should be organized using classes, functions, and header files for easier maintenance and understanding.
- Seen in: Flight class and AirlineBookingManager class

NFR5: Maintainability

- The program should separate functionalities into multiple files for easier updates and debugging.
- Seen in: AirlineBookingManager.h, Flight.h, main.cpp

NFR6: Security

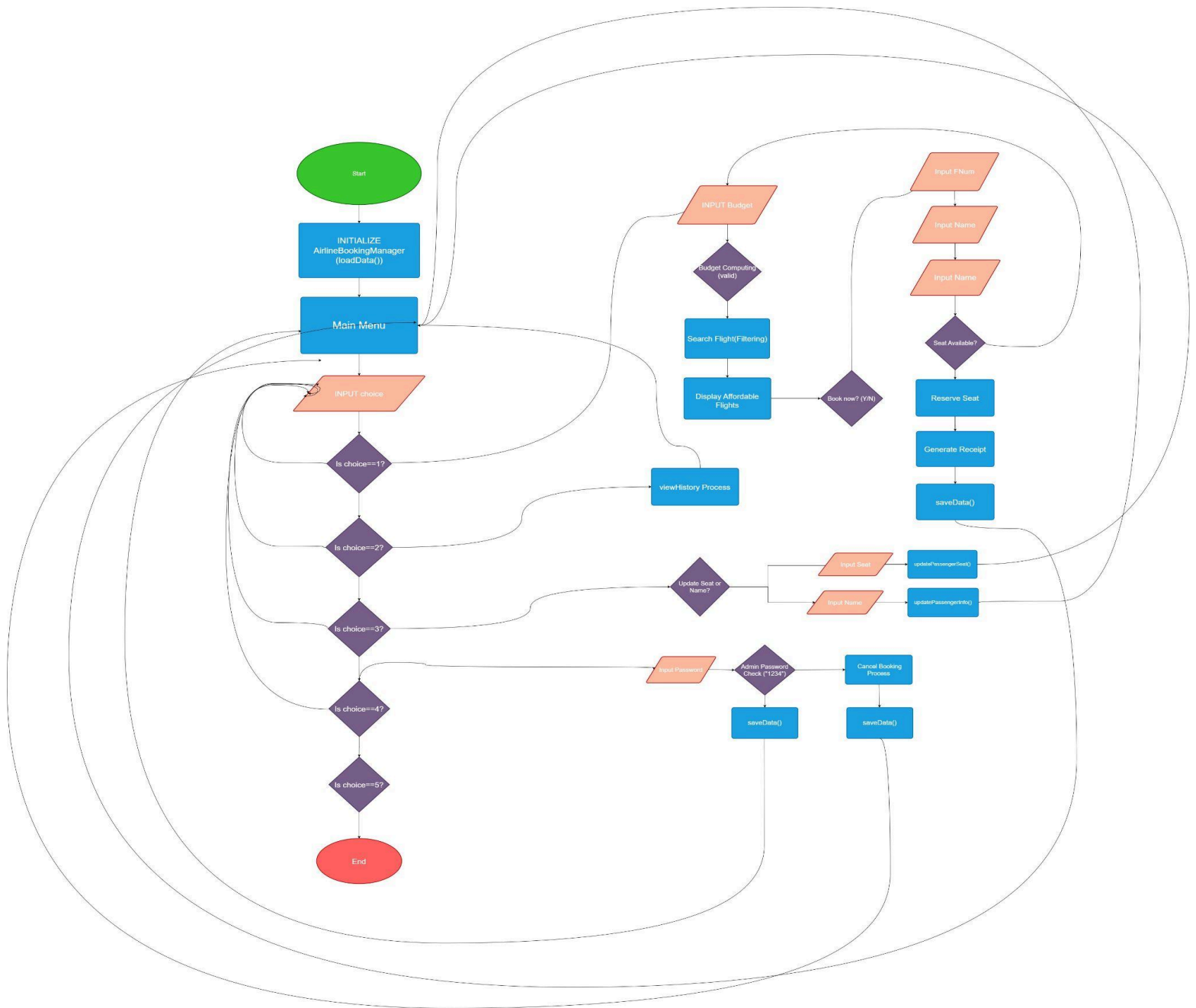
- The system should require an admin password before allowing booking cancellation.
- Seen in: cancelBooking()

NFR7: Performance

- The system should quickly display flights and process bookings using arrays and vectors.
- Seen in: vector<Flight>, vector<Transaction>

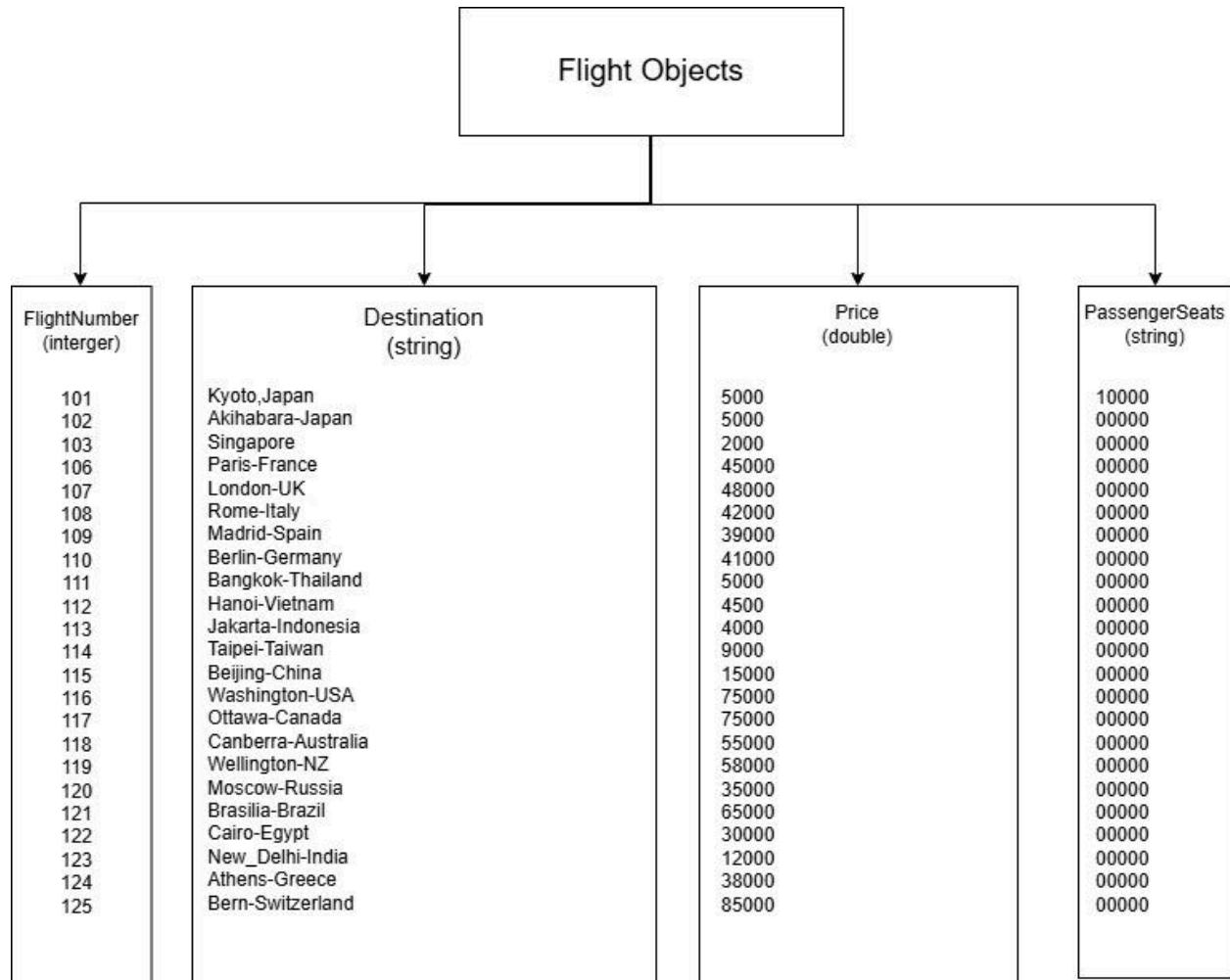
Section 3: Design Specification

Flowchart:



File Structure Diagram

For our system's saving mechanism, we are using a simple text file named **database.txt** located right inside the main project folder. This file makes sure all flight data stays saved even if you close the program. To keep things clean, we used a Space-Separated Values (Whitespace Delimiter) format, meaning we just used regular spaces to separate each piece of information in a line.



How Our Data Format Works:

When the program runs, it reads the **database.txt** file line by line. Each single line represents one Flight Object. We used the standard C++ extraction operator (>>) to automatically split the line into four different variables based on where the spaces are:

1. **flightNumber** (Integer): This is the very first number on the line (like 101, 102, or 103). It serves as the unique ID for each flight so they don't mix up. Since it's an integer, it only takes whole numbers.

2. **destination** (String): This is the second part of the line which holds the country or city name (like *Kyoto-Japan* or *Singapore*). We used a hyphen (-) for places with two words because the C++ >> operator automatically stops reading when it hits a blank space. The hyphen keeps the whole name together.
3. **price** (Double): The third number represents the ticket price for that flight (like 5000 or 2000). We chose the double data type here just in case we need to use exact decimals for pricing later on.
4. **Seats Data** (String): This is the 5-character block at the very end of the line (like 10000 or 00000). We specifically made this a string because if we used a regular integer, the computer would automatically delete the zeros at the beginning (leading zeros), which would break our layout.
 - Each position in the string (from index 0 to 4) matches Seat 1 to Seat 5 through our loadSeatData() function.
 - A 0 means the seat is Available (Vacant).
 - A 1 means the seat is Reserved (Occupied). For example, in Flight 101 (10000), it clearly shows that only Seat 1 is booked, while Seats 2, 3, 4, and 5 are still completely free.

References:

OpenAI. (2026). *Gemini* (Version 1.5 Flash) [Large language model].
<https://gemini.google.com>

hiliTomatoNoodle. (2018). *C++ Tutorial 16: File I/O (ifstream and ofstream)*.
<https://youtu.be/0NwsayeOsd4?si=1OSy8Fjz5QNWRxL->

Neso Academy. (2021). *Vectors in C++ Standard Template Library (STL)*.
https://youtu.be/dQG41m6vf3A?si=MTcfLo2mo_hArsHM

Simplilearn. (2022, January 11). Project in C++ | Airline registration system project in C++ | C++ project for beginners | Simplilearn

<https://youtu.be/ZXTwKLtER00?si=MkZYIC8TZfF9w95V>

